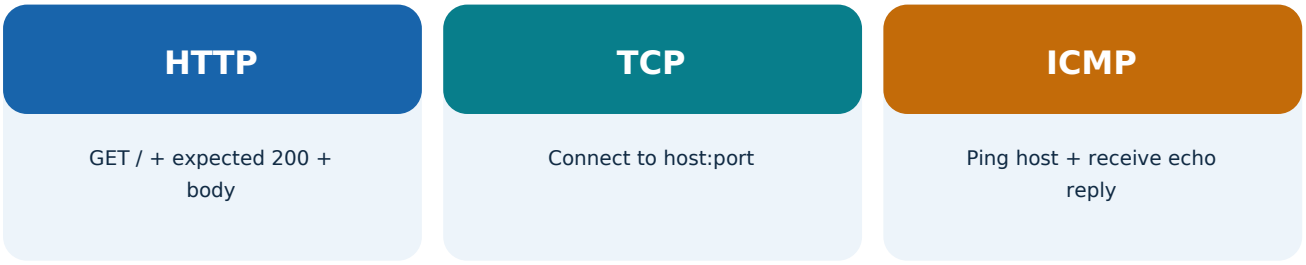
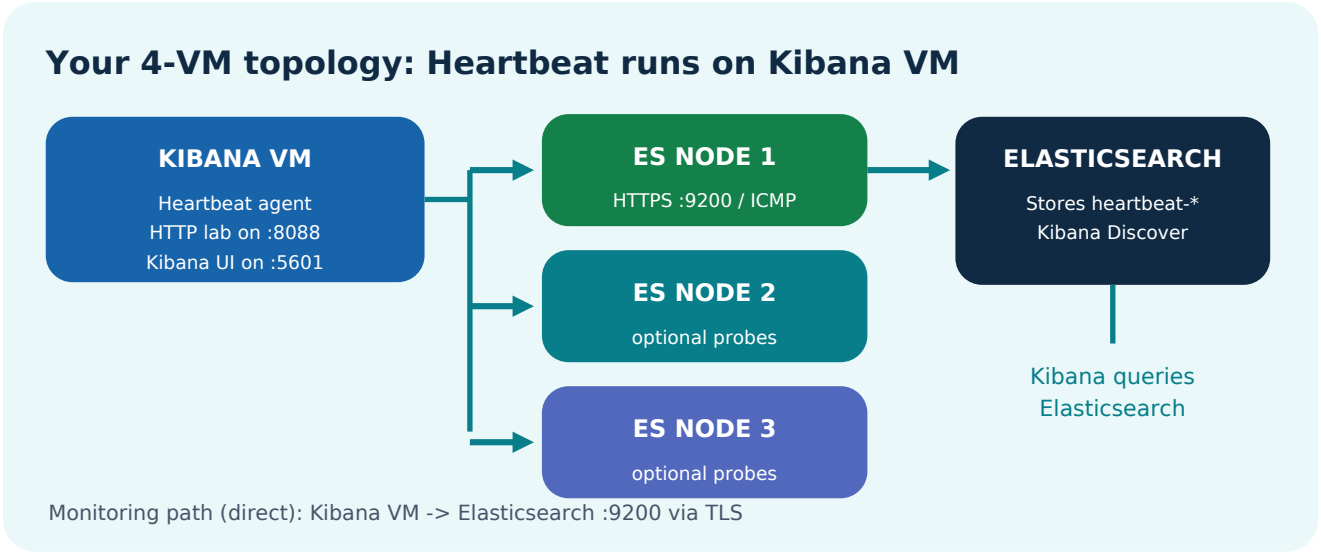
FIELD LAB MANUAL / ELASTIC STACK / UBUNTU

Heartbeat Uptime Monitoring

HTTP, TCP and ICMP | Installation, TLS configuration, testing and Kibana verification





Built for your lab: one Ubuntu Kibana VM, three Ubuntu Elasticsearch nodes (esnode-1, node2, node3), HTTPS/TLS-enabled Elasticsearch, the **existing Elasticsearch HTTP CA file on the Kibana VM**, and Kibana Dev Tools / Discover. Uses **nano** for configuration and APT for installation.

Goal: install one standalone Heartbeat agent on the Kibana VM; monitor an HTTP test web page, Kibana and Elasticsearch TCP ports, and ICMP reachability; publish results to Elasticsearch and demonstrate UP/DOWN evidence in Kibana.

Duration: approximately one guided lab session, depending on network and Elastic package availability. Commands are templates: replace the uppercase placeholders with values from your running VMs.

Important distinction: this is standalone Heartbeat, **not** Fleet-managed Elastic Agent and **not** browser-journey Synthetics. The legacy Uptime UI is deprecated; Kibana Discover is the reliable verification path.

1. Lab layout, prerequisites and checkpoints

The guide does not change your existing Elasticsearch cluster topology. Heartbeat is a monitoring **client** on the Kibana VM and writes check results directly to the Elasticsearch HTTPS endpoint. Its HTTP/TCP/ICMP checks are network probes; none require a new Elasticsearch node.

VM / Role	Already running	New changes
Kibana VM	Kibana on :5601; accesses ES	Install Heartbeat; create demo HTTP page on loopback :8088; monitor services
esnode-1	Elasticsearch HTTPS :9200	No installation or certificate copying required
node2	Elasticsearch HTTPS :9200	No changes; optional TCP/ICMP targets
node3	Elasticsearch HTTPS :9200	No changes; optional TCP/ICMP targets

What you need before beginning

1. Ubuntu shell with **sudo** access on the Kibana VM and the existing HTTP CA certificate already configured there for Kibana-to-Elasticsearch HTTPS.

2. A running Elasticsearch cluster and Kibana, with known Elasticsearch user password (training uses **elastic** only for initial setup).

3. A known node 1 DNS name or IP that is covered by the HTTPS certificate Subject Alternative Name (SAN).

4. Outbound network access from Kibana VM to Elasticsearch on TCP 9200, or working local DNS / **/etc/hosts** entries.

5. Internet/package repository access on the Kibana VM, and a compatible Elastic APT repository for the installed cluster major version.

Important: Do not insert raw passwords into heartbeat.yml. This manual stores the credential in the Heartbeat keystore. Never use `ssl.verification_mode: none` as the permanent solution to a certificate mismatch.

Learning outcomes

After this lab you can: verify agent health; explain why a TCP probe can be UP while an HTTPS application is unhealthy; trace **Heartbeat -> Elasticsearch -> Kibana**; filter monitor.status in Discover; and prove failure/recovery without stopping Elasticsearch.

2. Identify the Kibana VM and Elastic Stack release

Run on the Kibana VM (rps1 if this is your Kibana host). You do not need an Elasticsearch executable on this VM. Discover the Elastic release using the secured REST API after verifying the existing CA in Step 4.

```
hostnamectl
uname -m
lsb_release -a
systemctl is-active kibana
hostname -I
getent hosts esnode-1
```

Architecture: on most Intel/AMD PCs `uname -m` shows x86_64 (APT package architecture amd64); on ARM it may show aarch64 (APT architecture arm64). This guide uses APT, not manual package installation. The Elasticsearch version is confirmed in Step 4 using curl.

Expected evidence: Record the host name, Ubuntu version, CPU architecture and the actual Elasticsearch node name or IP. Do not install Elasticsearch on the Kibana VM just to determine its version.

3. Confirm network connections first

On Kibana VM, replace ESNODE1_NAME with the real certificate-valid DNS name or address. The TLS connection to Elasticsearch must work before Heartbeat can publish results.

```
getent hosts ESNODE1_NAME
nc -vz ESNODE1_NAME 9200
nc -vz 127.0.0.1 5601
ping -c 3 ESNODE1_NAME
```

If netcat is missing, use `sudo apt-get install -y netcat-openbsd`. A blocked ICMP echo from a server is not proof that TCP 9200 is unavailable. Record results separately.

Important: If a VM name does not resolve, map a name to its actual IP in **sudo nano /etc/hosts** on the Kibana VM. Use a hostname that is listed in the Elasticsearch HTTPS certificate; simply inventing an alias will not fix TLS identity verification.

4. Reuse the certificate already on your Kibana VM

Your Kibana VM already has an Elasticsearch HTTPS CA certificate. **Do not copy any certificate from esnode-1, node2 or node3.** The CA referenced by the Kibana setting `elasticsearch.ssl.certificateAuthorities` is the best starting point. It may be at `/etc/kibana/certs/http_ca.crt`, but use the path actually configured on your VM.

4A - Locate the existing certificate on the Kibana VM

```
sudo grep -n 'elasticsearch.ssl.certificateAuthorities' /etc/kibana/kibana.yml
sudo find /etc/kibana -type f -name '*.crt' -print
sudo find /etc/kibana -type f -name '*.pem' -print
sudo ls -l /etc/kibana/certs
```

The final listing command may say "No such file or directory" if the certificate is elsewhere. Use the exact path from **kibana.yml**, possibly in a YAML list. For example, if it lists `/etc/kibana/certs/http_ca.crt`, use that file. Do not guess the path.

4B - Inspect the existing CA

```
# CHANGE THIS EXAMPLE PATH to match YOUR kibana.yml.
KIBANA_CA=/etc/kibana/certs/http_ca.crt
sudo test -f "$KIBANA_CA" && echo 'CA file found'
sudo openssl x509 -in "$KIBANA_CA" -noout -subject -issuer -dates
```

If Kibana uses a CA bundle, use the CA bundle specified in its Elasticsearch configuration. Do not substitute a Kibana server certificate, an Elasticsearch private key, or a Fleet Server certificate.

4C - Make a local copy in the Heartbeat directory

```
# Use the same terminal as 4B, with KIBANA_CA set.
sudo mkdir -p /etc/heartbeat/certs
sudo install -m 0644 "$KIBANA_CA" /etc/heartbeat/certs/http_ca.crt
sudo ls -l /etc/heartbeat/certs/http_ca.crt
openssl x509 -in /etc/heartbeat/certs/http_ca.crt -noout -subject
```

The source CA stays in the Kibana directory; only the public CA is copied to `/etc/heartbeat/certs/http_ca.crt`. Run Step 4B again to reassign `KIBANA_CA` if you opened a fresh terminal.

4D - Use this CA to verify Elasticsearch and its version

```
# Replace ESNODE1_NAME with the real certificate-valid hostname/IP.
curl --cacert /etc/heartbeat/certs/http_ca.crt -u elastic https://ESNODE1_NAME:9200
```

Enter the Elasticsearch password at the curl prompt. JSON should include **"version": { "number": "X.Y.Z" }**. Record X.Y.Z for the APT installation. If a hostname mismatch appears, use the DNS name or IP included in the certificate Subject Alternative Name. Do not turn off certificate verification.

Expected evidence: Elasticsearch responds securely and the exact version X.Y.Z is recorded using the CA that was already on your Kibana VM.

5. Install Heartbeat on the Kibana VM using APT

Heartbeat should use the same Elastic **major** release as your existing cluster (8.x or 9.x) and preferably the same X.Y.Z version, when present in the APT repository. First inspect whether your Kibana VM already has the matching Elastic APT repository configured.

5A - Check for the existing Elastic package repository

```
sudo grep -R 'artifacts.elastic.co/packages' /etc/apt/sources.list.d/
apt-cache policy heartbeat-elastic
```

If an existing Elastic APT repository already matches your Elasticsearch major release, **skip Step 5B**. If none exists, add one. If several release lines exist, select the intended matching package version explicitly; do not accept the newest candidate blindly.

5B - If needed: add the correct repository using nano

```
sudo apt-get update
sudo apt-get install -y curl gnupg ca-certificates
curl -fsSL https://artifacts.elastic.co/GPG-KEY-elasticsearch -o /tmp/elastic-key
sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg /tmp/elastic-key
rm /tmp/elastic-key
sudo nano /etc/apt/sources.list.d/elastic-heartbeat.sources
```

Paste the following five lines into **nano** for the repository. This deb822 format keeps each line short and avoids copy/paste wrapping. Change **9.x** to **8.x** only if your Elasticsearch version begins with 8. Save with Ctrl+O, Enter, Ctrl+X.

```
Types: deb
URIs: https://artifacts.elastic.co/packages/9.x/apt
Suites: stable
Components: main
Signed-By: /usr/share/keyrings/elasticsearch-keyring.gpg
```

Important: If your Kibana VM already has a suitable Elastic source, do not create this second source file. If the signing key already exists, GPG may prompt before replacing it. The major release in URIs must match your existing cluster.

5C - Select and install the matching release

```
sudo apt-get update
apt-cache policy heartbeat-elastic
apt-cache madison heartbeat-elastic

# Replace VERSION_FROM_APT_LIST with a real version displayed.
sudo apt-get install -y heartbeat-elastic=VERSION_FROM_APT_LIST

heartbeat version
systemctl list-unit-files | grep heartbeat
```

Example: if Elasticsearch reports 9.5.3 and the package list includes 9.5.3, use `sudo apt-get install -y heartbeat-elastic=9.5.3`. If that exact release is unavailable, select a supported compatible version from the repository listing before proceeding. Do not install the newest package automatically without checking its version.

Expected evidence: The Heartbeat version is compatible with your running Elasticsearch cluster. The package installs `/etc/heartbeat/heartbeat.yml` and the service `heartbeat-elastic.service`.

6. Create an HTTP target for reliable UP/DOWN testing

This is a tiny local training endpoint hosted on the Kibana VM at **127.0.0.1:8088**. It avoids changing or interrupting Kibana and Elasticsearch during the demonstration. Use **nano**, not heredocs or **cat**.

6A - Install Python (if required), create the page

```
sudo apt-get update
sudo apt-get install -y python3
sudo mkdir -p /opt/heartbeat-lab/web
sudo nano /opt/heartbeat-lab/web/index.html
```

Inside nano, type or paste exactly:

```
<!DOCTYPE html>
<html>
<head><title>Heartbeat Ubuntu Lab</title></head>
<body>
  <h1>HEARTBEAT LAB OK</h1>
  <p>HTTP availability test on the Kibana VM.</p>
</body>
</html>
```

Save every nano file with **Ctrl+O**, **Enter**, **Ctrl+X**. Keep the text **HEARTBEAT LAB OK**; the monitor checks for it.

6B - Make the local test endpoint a systemd service

```
sudo nano /etc/systemd/system/heartbeat-lab-web.service
```

Paste:

```
[Unit]
Description=Heartbeat HTTP Training Target
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/heartbeat-lab/web
ExecStart=/usr/bin/python3 -m http.server 8088 --bind 127.0.0.1
Restart=on-failure
RestartSec=2

[Install]
WantedBy=multi-user.target
```

6C - Start and test your HTTP service

```
sudo systemctl daemon-reload
sudo systemctl enable --now heartbeat-lab-web
sudo systemctl status heartbeat-lab-web --no-pager
curl -i http://127.0.0.1:8088/
```

Expected evidence: HTTP 200 OK and the body contains HEARTBEAT LAB OK. If HTTP 8088 is occupied, choose another unused port in both the unit and the Heartbeat monitor.

7. Store the Elasticsearch password securely

Create the Heartbeat keystore as **root**, because the Debian systemd service runs Heartbeat with the Debian paths and must access the same keystore. Keep the explicit data and config paths.

```
sudo heartbeat keystore create \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo heartbeat keystore add ES_PWD \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo heartbeat keystore list \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
```

At the **add ES_PWD** prompt, enter the real **elastic** password. It should not be echoed. Confirm the key name is shown in the listing. Use a least-privilege dedicated publisher user later, once the lab works.

Expected evidence: The keystore contains a key named ES_PWD and no plaintext password has been written to the YAML file.

8. Configure HTTP, TCP and ICMP probes in nano

Back up the original package YAML, then fully replace its contents so the configuration is easy to read and indentation remains consistent.

```
sudo cp /etc/heartbeat/heartbeat.yml \
  /etc/heartbeat/heartbeat.yml.original
sudo nano /etc/heartbeat/heartbeat.yml
```

Paste the complete YAML below. Replace ESNODE1_NAME in both locations with the same certificate-valid host name you verified using curl. Use spaces, not tabs. The `127.0.0.1` entries refer to the Kibana VM, where Heartbeat runs.

```
heartbeat.monitors:
  - type: http
    id: kibana-vm-http-lab
    name: Kibana VM - HTTP Lab 8088
    schedule: '@every 10s'
    hosts: ["http://127.0.0.1:8088/"]
    check.response:
      status: [200]
      body:
        positive: ["HEARTBEAT LAB OK"]

  - type: tcp
    id: kibana-vm-tcp-5601
    name: Kibana VM - TCP 5601
    schedule: '@every 10s'
    hosts: ["127.0.0.1:5601"]

  - type: tcp
    id: esnode1-tcp-9200
    name: Elasticsearch node1 - TCP 9200
    schedule: '@every 10s'
    hosts: ["ESNODE1_NAME:9200"]

  - type: icmp
    id: kibana-vm-icmp-local
    name: Kibana VM - ICMP Loopback
    schedule: '@every 10s'
    hosts: ["127.0.0.1"]

  - type: icmp
    id: esnode1-icmp
    name: Elasticsearch node1 - ICMP
    schedule: '@every 10s'
    hosts: ["ESNODE1_NAME"]
```

Continue in the **same file**, with the following unindented top-level YAML sections:

```
heartbeat.scheduler:
  limit: 10

processors:
  - add_observer_metadata:
      geo:
        name: ubuntu-kibana-lab

output.elasticsearch:
  hosts: ["https://ESNODE1_NAME:9200"]
  username: "elastic"
  password: "${ES_PWD}"
  ssl.certificate_authorities:
    - "/etc/heartbeat/certs/http_ca.crt"

logging.level: info
```

Important: Do **not** add a Logstash output: this lab intentionally uses **direct Elasticsearch HTTPS**. The TCP check against 9200 verifies the port, not the Elasticsearch application or TLS health. The ICMP esnode1 probe may show DOWN if VM firewall/security rules block echo requests.

Why each monitor matters

ID	What a successful result means
kibana-vm-http-lab	HTTP returns 200 and the page body matches a required string.
kibana-vm-tcp-5601	A TCP connection can be established to the local Kibana port.
esnode1-tcp-9200	A TCP connection can be established to the Elasticsearch node HTTPS port.
kibana-vm-icmp-local	Loopback ICMP Echo Request receives a reply (requires ping privilege).
esnode1-icmp	Network path and ICMP policy allow ping to esnode-1.

9. Validate configuration and Elasticsearch output

Do not start the service until both validation commands succeed. Explicit paths ensure the command-line process finds the same **YAML** and **keystore** as systemd.

```
sudo chown root:root /etc/heartbeat/heartbeat.yml
sudo chmod 600 /etc/heartbeat/heartbeat.yml
sudo heartbeat test config -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo heartbeat test output -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
```

Expected evidence: Configuration test reports **Config OK**. Output test can connect to Elasticsearch and perform TLS/authentication successfully. An initial 401, x509, or connection error means do not proceed until corrected.

Special ICMP privilege check

On most Debian installations the service runs with sufficient privileges. If the HTTP and TCP checks are UP but ICMP reports **operation not permitted**, first inspect the service user and logs. Where necessary and appropriate, Elastic documents the Linux file capability:

```
systemctl cat heartbeat-elastic
sudo journalctl -u heartbeat-elastic -n 40 --no-pager
command -v getcap
sudo getcap /usr/share/heartbeat/bin/heartbeat
```

A non-root custom service may require `cap_net_raw` on the installed binary; do not change capabilities if running with adequate privileges. Confirm firewall ping rules when only a **remote** ICMP monitor is DOWN.

10. Initialize Elasticsearch assets and start the agent

Do one-time index/template setup with the training administrator credential, then run the systemd service. Only Elasticsearch index-management assets are required for data ingestion; this avoids dependence on Kibana dashboard setup permissions or legacy Uptime assets.

```
sudo heartbeat setup --index-management -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo systemctl enable --now heartbeat-elastic
sudo systemctl restart heartbeat-elastic
sudo systemctl status heartbeat-elastic --no-pager
sudo journalctl -u heartbeat-elastic -n 60 --no-pager
```

Look for events published successfully and no repeated errors. Heartbeat will immediately begin running the five probes on their 10-second schedules.

Expected evidence: `systemctl is-active heartbeat-elastic` returns **active**; journal logs do not show repeated publishing failures. After approximately a few probe cycles there should be documents under heartbeat-* in Elasticsearch.

11. Check data in Kibana Dev Tools

On your browser, open the existing Kibana URL and navigate to **Dev Tools -> Console**. Run the requests below, one request at a time. Do not enter shell commands into Console.

```
GET _data_stream/heartbeat-*

GET heartbeat-*/_count

GET heartbeat-*/_search
{
  "size": 10,
  "sort": [{ "@timestamp": { "order": "desc" } }],
  "_source": [
    "@timestamp", "monitor.id", "monitor.name",
    "monitor.type", "monitor.status", "observer.geo.name",
    "url.full", "error.message"
  ]
}
```

Expected evidence: Count increases after new probes; search results have values such as **monitor.type = http/tcp/icmp** and **monitor.status = up**. The optional remote ICMP check can be DOWN if ping is blocked.

12. Check results in Discover using the heartbeat-* data view

1. Open Kibana and select **Discover** from the navigation menu.

2. Use an existing **heartbeat-*** data view if it exists. Otherwise create it under **Stack Management -> Data Views -> Create data view**.

3. Set the name/pattern to **heartbeat-***; use **@timestamp** as the time field. Save.

4. In Discover, choose the heartbeat-* data view and set time range to **Last 15 minutes** (or Last 1 hour for a slow lab).

5. Add columns **@timestamp**, **monitor.name**, **monitor.type**, **monitor.status**, and optionally **error.message**.

6. Paste one of the KQL examples below in the query bar; press Refresh.

KQL practice

```
monitor.status : "up"
monitor.status : "down"
monitor.type : "http"
monitor.type : "tcp"
monitor.type : "icmp"
monitor.id : "kibana-vm-http-lab"
monitor.name : "Elasticsearch node1 - TCP 9200"
```

KQL is entered into the **Discover query bar**, not Dev Tools. A small count of results is normal for a short time window. Increase the time window and verify the monitor data view before troubleshooting further.

Expected evidence: Discover shows one or more documents for each configured monitor, with timestamp, type and UP/DOWN status. Save a Discover search named **Heartbeat - Lab Monitor Status**.

13. Lab exercise A: prove HTTP DOWN and recovery

This exercise only stops the **training HTTP server** on port 8088. Do not stop Kibana, Elasticsearch, Logstash, or Fleet Server.

A1 - Baseline UP

```
curl -I http://127.0.0.1:8088/
systemctl is-active heartbeat-lab-web
```

Expected: HTTP/1.0 or HTTP/1.1 200; the HTTP Heartbeat monitor is UP in Discover.

A2 - Intentionally stop the test target

```
sudo systemctl stop heartbeat-lab-web
curl -I http://127.0.0.1:8088/
```

Curl should fail to connect. Allow a couple of 10-second monitor runs, then in Discover query:

```
monitor.id : "kibana-vm-http-lab" AND monitor.status : "down"
```

Expected: DOWN event for the HTTP monitor. **Other monitors remain UP** because they check different services.

A3 - Recover the target

```
sudo systemctl start heartbeat-lab-web
curl -I http://127.0.0.1:8088/
```

```
monitor.id : "kibana-vm-http-lab" AND monitor.status : "up"
```

Expected evidence: Save evidence of at least one DOWN event and a newer UP event, proving recovery. Screenshot the Discover table and record both timestamps.

14. Lab exercise B: prove TCP closed-port detection

Do not stop Elasticsearch port 9200. Instead add a temporary TCP monitor against a port that is not listening on localhost. Check that the test port is unused:

```
ss -lnt | grep ':65534' || true
sudo nano /etc/heartbeat/heartbeat.yml
```

Under **heartbeat.monitors**, add this list item **before** the `heartbeat.scheduler:` section (use two-space indentation for the dash, like other monitors):

```
- type: tcp
  id: kibana-vm-tcp-closed
  name: Kibana VM - Closed TCP Port
  schedule: '@every 10s'
  hosts: ["127.0.0.1:65534"]
```

```
sudo heartbeat test config -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo systemctl restart heartbeat-elastic
sudo journalctl -u heartbeat-elastic -n 30 --no-pager
```

In Discover query:

```
monitor.id : "kibana-vm-tcp-closed" AND monitor.status : "down"
```

Expected evidence: The deliberately closed TCP port is DOWN. The Kibana port 5601 and Elasticsearch port 9200 monitors should remain UP. Remove the temporary monitor from nano after capturing evidence, then restart Heartbeat.

15. Lab exercise C: test ICMP locally and remotely

C1 - Baseline network commands

```
ping -c 3 127.0.0.1
ping -c 3 ESNODE1_NAME
sudo journalctl -u heartbeat-elastic -n 40 --no-pager
```

Look for the ICMP loopback monitor in Discover:

```
monitor.id : "kibana-vm-icmp-local"
```

If the remote ping fails, test TCP 9200 separately. An ICMP block is common when firewalls disallow Echo Request/Reply; it does not necessarily indicate an Elasticsearch outage.

C2 - Optional remote ping policy demonstration

For an isolated training VLAN only, an instructor may temporarily change a VM firewall rule to block ICMP; this is **optional**. Do not change enterprise firewall or production security policy. A safer alternative is to monitor an instructor-approved nonresponsive test host, while documenting that this is an ICMP reachability result only.

Expected evidence: Explain the difference: ICMP = host responds to ping; TCP = port accepts a connection; HTTP = endpoint returns the expected application response.

16. Optional: monitor Elasticsearch HTTP/TLS, not just TCP

The **TCP :9200** monitor only checks socket availability. An additional HTTPS monitor can validate that Elasticsearch responds at the application layer and that its certificate chains to the expected CA.

Add this to the **heartbeat.monitors** list via nano. An unauthenticated secured Elasticsearch endpoint typically returns **401**; this is an intentional and valid example of an expected response. If your environment behaves differently, inspect with curl and adjust the expected code accordingly.

```
- type: http
  id: esnode1-https-9200
  name: Elasticsearch node1 - HTTPS TLS
  schedule: '@every 30s'
  hosts: ["https://ESNODE1_NAME:9200/"]
  ssl.certificate_authorities:
    - "/etc/heartbeat/certs/http_ca.crt"
  check.response:
    status: [401]
```

Validate and restart Heartbeat as in Step 14. Do **not** set `check.response.status: [200]` unless the HTTPS target actually returns 200 for that request. A 401 here means the secured service is reachable and asking for authentication.

17. Optional: add the other two Elasticsearch nodes

To include node2 and node3 without installing Heartbeat on them, add additional **TCP monitors** to the same Heartbeat YAML list. Only add nodes that can be reached from the Kibana VM.

```
- type: tcp
  id: esnode2-tcp-9200
  name: Elasticsearch node2 - TCP 9200
  schedule: '@every 10s'
  hosts: ["ESNODE2_NAME:9200"]

- type: tcp
  id: esnode3-tcp-9200
  name: Elasticsearch node3 - TCP 9200
  schedule: '@every 10s'
  hosts: ["ESNODE3_NAME:9200"]
```

The TCP checks do not perform Elasticsearch authentication, and therefore do not require the Elasticsearch CA themselves. Only the **output.elasticsearch** configuration uses CA trust for forwarding Heartbeat events.

Check and restart

```
sudo heartbeat test config -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo systemctl restart heartbeat-elastic
```

```
monitor.id : "esnode2-tcp-9200"
monitor.id : "esnode3-tcp-9200"
```

Expected evidence: Discover reports each node separately under its own monitor.id. This lets you compare individual node port reachability from the Kibana VM.

18. Create a lightweight Kibana dashboard

In modern versions, the older Uptime interface can be hidden or deprecated. For your training, build a simple **Lens dashboard** using the Heartbeat data view, so it works regardless of whether legacy Uptime is visible.

1. Kibana -> **Analytics -> Dashboard** -> **Create dashboard** -> **Create visualization**.

2. Select **heartbeat-*** as the data view and choose **Lens** (or a similar current visualization editor).

3. For a **Monitor status** chart, choose **Count of records** with the **Top values of monitor.name** as a breakdown. Filter **monitor.status : "down"** to see failures only.

4. For a **status-over-time** chart, use **@timestamp** on the horizontal axis, **Count of records** as the metric and **monitor.status** as the series/split.

5. Add a second visualization showing **Count by monitor.type** (HTTP, TCP, ICMP) and save the dashboard as **Ubuntu Heartbeat Availability Lab**.

6. Set the dashboard time range to **Last 30 minutes**; perform the HTTP outage experiment and refresh.

Expected evidence: Dashboard shows monitor events; the DOWN series increases when the HTTP test server is stopped and the UP series increases after recovery.

19. Optional alert creation (connectors not required for core lab)

If your Kibana version and license support the rule type, create an **Elasticsearch query** or **document threshold** rule against heartbeat-* with `monitor.status: down` and a short evaluation window. A rule without a configured action connector may still be useful for rule-state demonstration, but actual email/webhook notifications require a working connector. Since connectors were previously disabled in your environment, treat email/webhook as a separate exercise.

Avoid alerting solely on one remote ICMP failure when network policy intentionally blocks ping. Choose the controlled **kibana-vm-http-lab** monitor as your initial target.

20. Troubleshooting decision table

Symptom	Most likely cause / next check
heartbeat command not found	Check command -v heartbeat and apt-cache policy heartbeat-elastic; install the matching APT version.
Config error or YAML unexpected indent	Only spaces, no tabs; ensure list dashes align; run heartbeat test config before restart.
401 Unauthorized for output	Wrong elastic password/keystore key, bad permissions, or an account without required privileges.
x509 unknown authority	CA absent, wrong CA, or copied a Fleet/Kibana CA instead of Elasticsearch HTTP CA.
x509 name mismatch	Use the hostname/IP listed on certificate SAN and update DNS or /etc/hosts as needed.
connection refused :9200	Wrong ES host/IP, stopped Elasticsearch, listener or firewall issue; verify nc -vz and curl.
HTTP lab :8088 DOWN	systemctl status heartbeat-lab-web; curl -i localhost:8088; expected body text.
TCP Kibana :5601 DOWN	Kibana may be bound to different interface, is stopped, or port changed; check ss -lnt.
ICMP local DOWN	Probe privilege missing; inspect journal and service user/capabilities.
Remote ICMP DOWN but TCP UP	Firewall or cloud/network policies block ICMP; not proof of server outage.
No heartbeat-* data view	Create data view heartbeat-*, time field @timestamp; or verify documents exist first.
No new documents	Check systemd agent status, test output, logs, credentials, TLS and time range.
Legacy Uptime menu absent	Expected on newer stacks; use Discover. The older UI is deprecated/conditionally hidden.

Quick diagnostic commands, Kibana VM

```
systemctl is-active heartbeat-elastic
systemctl is-active heartbeat-lab-web
sudo journalctl -u heartbeat-elastic -n 100 --no-pager
sudo journalctl -u heartbeat-lab-web -n 30 --no-pager
sudo heartbeat test config -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
sudo heartbeat test output -e \
  --path.config /etc/heartbeat --path.data /var/lib/heartbeat
ss -lnt | grep -E ':(5601|8088|9200)'
```

Important: For failures, start from the lowest layer: DNS -> network TCP -> TLS trust/hostname -> authentication -> YAML -> service -> Elasticsearch documents -> Kibana data view/time filter. Avoid random edits to three Elasticsearch nodes.

21. Lab completion checklist and evidence

1. Heartbeat package version matches a compatible Elasticsearch stack release and heartbeat-elastic service is active.

2. Existing Kibana CA verified and copied locally; HTTPS curl works **without -k**.

3. Heartbeat keystore ES_PWD created; credentials are not hardcoded.

4. heartbeat.yml contains HTTP, TCP and ICMP monitors and passes test config/output.

5. Elasticsearch contains new heartbeat-* documents with monitor.type and monitor.status.

6. Discover data view exists; monitor.name, monitor.id, monitor.type and monitor.status columns are visible.

7. HTTP experiment shows both DOWN and recovered UP events.

8. Closed-port TCP experiment shows DOWN without impacting Elasticsearch.

9. ICMP localhost is verified; remote ICMP exceptions are explained rather than misdiagnosed.

10. Optional dashboard saved with status trends and type breakdown.

Evidence to collect: screenshot 1 - systemctl active; screenshot 2 - heartbeat test output; screenshot 3 - Dev Tools count/search; screenshot 4 - Discover UP records; screenshot 5 - HTTP DOWN; screenshot 6 - HTTP UP recovery; screenshot 7 - TCP closed-port DOWN; screenshot 8 - dashboard.

22. Cleanup / reset training-only changes

Keep Heartbeat installed if you plan to continue into alerting and dashboards. Remove only the intentionally added failure monitor in the YAML and restart. To delete the optional HTTP training endpoint after the course, stop and disable its service, remove the dedicated unit/file and reload systemd.

```
# Optional: after the course; do not delete ES/Kibana files.
sudo systemctl disable --now heartbeat-lab-web
sudo rm /etc/systemd/system/heartbeat-lab-web.service
sudo systemctl daemon-reload
sudo rm -rf /opt/heartbeat-lab
```

Important: If you remove the HTTP demo server, also remove or disable its monitor in heartbeat.yml; otherwise Heartbeat will keep correctly reporting it as DOWN.

23. Instructor-led discussion prompts

1. HTTP vs TCP: Can TCP port 9200 be UP while Elasticsearch still returns unauthorized or an application error? Explain why.

2. ICMP vs TCP: Why can a remote node ignore ping while HTTPS is operational?

3. Direct output: What is the role of the output.elasticsearch HTTPS CA? Why is the HTTP lab endpoint a different destination?

4. Architecture: Why is one Heartbeat monitoring host sufficient to probe multiple Elasticsearch nodes?

5. Modern UI: Why use Discover for standalone Heartbeat instead of relying on the Synthetics browser-journey interface?

6. Production hardening: What would change to move from elastic superuser to a dedicated publisher role, redundant monitoring hosts, secure alerting, and a measured SLA?

24. Official Elastic references and version notes

Documentation and behavior are version-dependent. Confirm UI menu labels and package version against the stack that is actually installed. The following are official references used when preparing this manual:

1. Heartbeat installation / quick start
2. Monitor YAML configuration
3. HTTP monitor options
4. TCP monitor options
5. ICMP monitor permissions
6. Elasticsearch output and TLS
7. Keystore
8. APT repository setup
9. systemd unit
10. Legacy Uptime deprecation